4 Email, keys and AI

FULL

17. Your Android app, and coding it with AI

Quick version

No time to read it all? This is the short version. The full guide is below.

- Build the Jun's Garden Android app on your Windows desktop with Android Studio, run it on your phone, and use AI to help change the code safely.
- About 3 hours, split over a few sessions. Free.
- Part A: install Android Studio, open the `android-app` folder, switch on USB debugging on your phone, and click the green **Run** triangle.
- Part B: switch on Gemini in Android Studio (the free one is "Gemini for individuals").
- Part C (optional): a local AI with Ollama. Start with `qwen2.5-coder:7b`.
- Part D, every change: save first (git), ask for ONE small change, read it with `git diff`, build, test, then keep it (commit) or undo it (`git restore .`).
- If your phone already has the app from the website, Android may refuse to install your build (the signing key). Do **not** uninstall the app from family phones to "fix" this.
- Never paste a password, a key file, `.env` or `local.properties` into any AI. The free Gemini may use what you type to train it.

Level: medium. **Time:** about 3 hours, split over a few sessions. **Cost:** free.

What you will build

- **Android Studio** on your Windows desktop, with the Jun's Garden app open in it.
- The app **running on your phone** (or on a pretend phone on the screen), built by you.
- **Gemini in Android Studio**, Google's AI helper, switched on, so you can ask it to explain code and suggest changes.
- Optional: a **local AI** on your RTX 4070 Ti, so you can ask coding questions without anything leaving your computer.
- A safe habit for **changing code with AI**: save first, one small change, read it, build it, test it, keep it or undo it.

The app's own guide is `android-app/README.md`. It explains every screen, where the data comes from, and how updates and signing work. Keep it open next to this guide.

Time and cost

Part	Time	Cost	Download size
A. Android Studio and the app	1 to 1.5 hours (most of it waiting for downloads)	free	Android Studio about 1.5 GB, plus Android tools it fetches
B. Gemini in Android Studio	15 minutes	free (with limits)	none
C. Local AI (optional)	30 to 45 minutes	free	about 5 to 9 GB per model
D. The AI coding habit	30 minutes, then every time	free	Git for Windows, small

What you need first

- Your Windows desktop (the one with the RTX 4070 Ti). Android Studio is big. Check the **system requirements** on Google's install page before you start:

https://developer.android.com/studio/install. For the pretend phone (emulator), Google recommends 16 GB of memory (RAM) and 16 GB of free disk space: https://developer.android.com/studio/run/emulator.

- The `android-app` folder from your garden package. Copy it to your desktop computer, for example to `C:\Users\<you>\junsgarden-app\`.
- An Android phone and its USB cable (or use the emulator instead).
- A Google account (for Gemini).
- Your password manager.

Part A. Android Studio, and the app on your phone

Steps

1. **Download Android Studio** from the official page: https://developer.android.com/studio. Click the big download button and accept the terms. (In October 2026 the newest version was called "Rabbit", 2026.2.1. The name changes with each new version; that is fine.)

You should see: a file like `android-studio-...-windows.exe` in your Downloads. 2. **Install it.** Double-click the file and follow the setup wizard. Keep the suggested choices. The first time Android Studio starts, it offers to download the Android tools (the "SDK"). Keep the suggested choices and let it finish. This is the longest wait.

You should see: the "Welcome to Android Studio" window. 3. **Open the app.** Click **Open**, and choose the `android-app` folder (the one with `settings.gradle.kts` inside). If it asks whether you trust the project, say yes: this is your own code. 4. **Wait for the Gradle sync.** Gradle is the tool that builds the app. The first time, it downloads everything the app needs. Watch the bar at the bottom. If you ever see a bar saying the project needs syncing, click **Sync Now** (official page: https://developer.android.com/studio/build).

You should see: no red errors, and on the left a tree with `app`, `kotlin+java`, `res`. Yellow warnings are fine (the README explains them). 5. **Get your phone ready (USB debugging).** On the phone: **Settings > About phone**, tap **Build number** seven times. A new **Developer options** menu appears. Open it and switch on **USB debugging**. Plug the phone into the computer. On the phone, allow the computer when it asks. Official page: https://developer.android.com/studio/run/device (it also links to USB drivers, if Windows does not see the phone).

You should see: your phone's name in the device list at the top of Android Studio.

No phone? Use the emulator (a pretend phone in a window). Make a virtual device as the official page shows, then pick it in the device list at the top. The first start can take a minute. Official page: https://developer.android.com/studio/run/emulator 6. **Run the app.** Click the green **Run** triangle at the top.

You should see: the garden film, then the sign-in screen, on the phone. Tap **Continue as guest** and scroll the plants.

Important: if your phone already has Jun's Garden from the website, Android may say the app "conflicts" or cannot be installed. That is the **signing key** rule (README, "Signing and updates"). Use the emulator, or a phone without the app, until you have decided what to do about the key. Do **not** uninstall the app from family phones to "fix" this. 7. **Build an APK file.** Menu **Build > Generate Bundle(s) / APK(s) > Generate APK(s)** (older versions: **Build > Build Bundle(s) / APK(s) > Build APK(s)**). Official page: https://developer.android.com/build/build-for-release

You should see: a message with a **locate** link, opening `app\build\outputs\apk\debug\` with `app-debug.apk` inside.

The same from a terminal (the **Terminal** tab at the bottom of Android Studio): `.\gradlew.bat assembleDebug`. Official page: https://developer.android.com/build/building-cmdline

How you know it worked

- The app opens on your phone or the emulator, and you can see your plants.
- `app-debug.apk` exists in `app\build\outputs\apk\debug\`.

Part B. Gemini in Android Studio

What it is

Gemini in Android Studio is an AI helper built into Android Studio. You type questions in a chat panel, next to your code. Google lists what it can do: write code, fix Gradle build errors, help with crashes, generate unit tests, write commit messages, explain and document code, and more. Official overview: https://developer.android.com/studio/gemini/overview

The free version is a "lightweight version of Gemini with a small context window": it can look at less of your project at once than the paid versions.

Steps

1. In Android Studio, open the AI panel: click **Agent** on the side bar, or **View > Tool Windows > Agent**.
2. Click **Sign in** and sign in with your Google account.
3. Choose **Gemini for individuals** (the free one).
4. It asks about **sharing your project with Gemini**. "Use all Gemini features" lets it read your project files, which makes its answers much better. Read part "Privacy" below first, then choose. Official steps: https://developer.android.com/studio/gemini/get-started
5. Try it. Open `MainActivity.kt` and ask: "Explain what this file does in simple words, step by step."

You should see: an answer that mentions the splash screen, the sign-in screen and the garden screen.

Privacy: what Gemini can see

- The setting is in **File > Settings > Tools > Gemini**. You can choose to decide **per project**.
- **The free version may use what you type in the chat to train Gemini.** Google's page says: "The code you explicitly enter into the chat experience may be used to train Gemini." So never type or paste anything private.
- You can hide files from Gemini with a file called `.aiexclude`. It works like `.gitignore` (one pattern per line). Put one in the `android-app` folder, listing at least: `*.jks`, `*.keystore`, `local.properties`. Writing it is a good first job for you. Official pages: https://developer.android.com/studio/gemini/data-and-privacy and https://developer.android.com/studio/gemini/aiexclude

Free, and its limits

The free tier has limited use per day and a smaller context window. Paid options (an API key, Google One AI plans, or business plans) have higher limits. You do not need them. Official comparison: https://developer.android.com/studio/gemini/feature-comparison

Good questions to ask it

- "Explain this function line by line in simple English."
- "This build error appeared after I changed X. What does it mean? Show the smallest fix, and explain it before changing anything."
- "Where in this project is the website address set?"
- "Write a unit test for `sunIcon` in `PlantIcons.kt`."

Part C. Local AI on your RTX 4070 Ti (optional)

What it is

A **local model** is an AI that runs on your own computer, on the graphics card. Nothing you type leaves your home. It is free once downloaded. **Ollama** is the program that runs the models.

Android Studio can use a local model in its Gemini chat. This was added in **Android Studio Otter 3 Feature Drop** (January 2026): https://developer.android.com/blog/posts/llm-flexibility-agent-mode-improvements-and-new-agentic-experiences-in-android-studio-otter-3-feature-drop

Be honest with yourself about what it can do. Google's own page says local models are "usually less accurate and have higher latency" (slower) than Gemini in the cloud, and that many Android Studio AI features do not work with a local model. The chat works. Use local AI for private questions and practice; use Gemini (part B) when you want the best answer.

Which models fit in 12 GB of graphics memory

Your RTX 4070 Ti has 12 GB of graphics memory (VRAM). Ollama's GPU page lists the 4070 Ti as supported: https://docs.ollama.com/gpu. A model runs fast when it fits in the VRAM. These sizes are the downloads shown on Ollama's pages in October 2026:

Model	Download size	Fits in 12 GB?	Page
`qwen2.5-coder:7b`	4.7 GB	yes, with room to spare	https://ollama.com/library/qwen2.5-coder/tags
`qwen2.5-coder:14b`	9.0 GB	yes, but tight	same page
`gemma4:e4b`	6.6 GB	yes. Google suggests Gemma 4 for Android Studio	https://ollama.com/library/gemma4/tags

Model	Download size	Fits in 12 GB?	Page
`gemma4:12b`	about 8 GB	yes	same page
`qwen3-coder:30b`	19 GB	no: spills into normal memory, much slower	https://ollama.com/library/qwen3-coder/tags
`gpt-oss:20b`	14 GB	no, the same	https://ollama.com/library/gpt-oss/tags

The AI also needs some VRAM for the conversation itself (the "context"). A longer context needs more. So start with a 7B model.

Steps

1. **Install Ollama for Windows**: https://ollama.com/download/windows. It needs Windows 10 22H2 or newer and an NVIDIA driver 551.61 or newer: https://docs.ollama.com/windows. (Guide 15 also covers Ollama.)

You should see: the Ollama icon in the system tray. 2. **Download a model.** In PowerShell:

```
ollama pull qwen2.5-coder:7b ollama run qwen2.5-coder:7b "Explain what a Kotlin data class is, in simple words."
```

You should see: an answer within a few seconds. 3. **Give it a bigger context.** Android Studio's page says to start the model "with a sufficiently large context length". Ollama's default on a 12 GB card is 4,000 tokens, which is small. In the Ollama app, open **Settings** and move the **context length** slider up (try 16k first). Then run `ollama ps` while a model is loaded: the CONTEXT column shows the size, and it should say it is using the GPU. If it says CPU, pick a smaller context or a smaller model. Official page: https://docs.ollama.com/context-length 4. **Connect it to Android Studio. File > Settings > Tools > AI > Model Providers**. Click **+** (add), choose **Local Provider**, type a description (`Ollama`), set the port to **11434** (Ollama's address is `http://localhost:11434`: https://docs.ollama.com/api/introduction), and enable the model you downloaded. Official

page: https://developer.android.com/studio/gemini/use-a-local-model 5. **Pick it.** In the Gemini chat window, use the **model picker** to switch from Gemini to your local model.

You should see: answers that come from your own computer. Switch the model picker back to Gemini at any time.

Another option (not needed): the **Continue** plugin can also connect an IDE to Ollama (https://docs.continue.dev/customize/model-providers/top-level/ollama). Its JetBrains plugin lists Android Studio as compatible, but the plugin page calls it "community-maintained". The built-in way above is simpler, so use that first.

Part D. The safe way to change code with AI

AI is fast, and sometimes wrong in a confident voice. This habit makes every change small, visible, and easy to undo.

One-time setup: Git for Windows

1. Install Git: https://git-scm.com/install/windows. Keep the suggested choices.
2. Tell Git your name (in the Android Studio **Terminal** tab):

```
git config --global user.name "Jun" git config --global user.email "you@example.com"
```

3. Save the starting point of the app (in the `android-app` folder):

```
git init git add . git commit -m "The app as I received it"
```

The `.gitignore` file already keeps build files and key files out.

You should see: a line with `files changed`.

Every change, the same 6 steps

1. **Save first.** `git status` should say "nothing to commit". If not, commit what you have. For a bigger idea, make a branch: `git switch -c try-bigger-plant-names` (official page:

https://git-scm.com/docs/git-switch).

2. **Ask for ONE small change.** For example: "In `CatalogScreen.kt`, make the plant name text one size bigger. Change nothing else. Explain the change before you make it."
3. **Read the change.** `git diff` shows every line that changed (red = removed, green = added). If you do not understand a line, ask the AI to explain that line. If it changed files you did not ask about, be suspicious.
4. **Build it.** Click **Run**, or `.\gradlew.bat assembleDebug`.
5. **Test it on the phone or emulator.** Look at the screen you changed, and at one screen you did not change.
6. **Keep it or undo it.** - Keep: `git add .` then `git commit -m "Bigger plant names"`. - Undo: `git restore .` puts changed files back as they were at your last commit (https://git-scm.com/docs/git-restore). New files the AI made are listed by `git status` under "Untracked"; delete them by hand. - On a branch and want to go back: `git switch -`.

Safety rules (always)

- **Never paste** a password, the garden login, a key file (`.jks`, `.keystore`), `.env`, or `local.properties` into any AI. This includes Gemini and local AI. The free Gemini may use what you type to train it.
- **Never let AI change** `applicationId` or `versionCode` in `app/build.gradle.kts` unless you are publishing on purpose (README, "Publish a new version").
- **AI can be wrong.** For anything about signing, passwords, the internet or permissions, check the official page it is based on. If AI and an official page disagree, trust the official page.
- **Small steps.** One change, one build, one test. If something breaks, you know exactly which change did it.

Part E. How the app talks to your website (in 5 lines)

1. `GARDEN_BASE` in `app/build.gradle.kts` is your website address (`https://junsgarden.zapto.org`). Change it there if the site ever moves.
2. Plants, beds and barcodes come from `GARDEN_BASE/app_api.php?action=...` (`list`, `detail`, `bed_counts`, `barcode_lookup`). No login needed.
3. Logging in uses `app_api.php?action=login`, `whoami` and `logout`. The app keeps only a key, never the password.
4. The camera tools send photos to `GARDEN_BASE/app/garden/...` (`identify`, `diagnose`, `tag`, `tag-queue`, `receipt`) with that key.
5. Updates come from `site/apps/version.json` and `site/apps/junsgarden.apk`. Details: `android-app/README.md`.

How you know it worked

- The app runs on your phone or the emulator from your own build.
- Gemini answers a question about `MainActivity.kt`.
- (Optional) Your local model answers in the same chat, with the internet switched off.
- You made one small change, saw it in `git diff`, tested it, and kept it with a commit (or undid it with `git restore .`).

If something goes differently

What you see	What to do
Gradle sync fails with "SDK location not found"	The Android tools are not installed yet. Finish Android Studio's setup wizard (it downloads them), or click the fix link Android Studio shows in the error, then **Sync Now**.

What you see	What to do
"Gradle JDK" or Java errors	**File > Settings > Build, Execution, Deployment > Build Tools > Gradle**, set Gradle JDK to the one that comes with Android Studio (JetBrains Runtime, "jbr"). For the terminal, set `JAVA_HOME` to the same one (README, "Build a debug APK"). Official page: https://developer.android.com/build/jdks
Phone not in the device list	Check USB debugging is on, try another cable, accept the prompt on the phone. The device page links to USB drivers.
"App not installed" or "conflicts with an existing package"	The signing key is different from the app already on that phone. Read README, "Signing and updates". Use the emulator meanwhile.
Gemini says you are not signed in, or is not offered	Sign in again in the Agent panel. Gemini needs a recent Android Studio version; update Android Studio.
The local model is very slow	`ollama ps`: if it shows CPU, the model or context is too big for 12 GB. Use `qwen2.5-coder:7b` or a smaller context.
The local model is not in the model picker	Check Ollama is running (tray icon), the port is 11434, and the model is enabled under Model Providers.
The AI's change does not build	Paste the **exact** red error text into the AI and ask it to explain first. Or undo with `git restore .` and ask for a smaller change.

Make it yours

- Make the two updater addresses in `Updater.kt` use `GARDEN_BASE` (README, "Change the website address").
- Remove one unused library (README, "Known leftovers"), build, and see that nothing broke.
- Remove the leftover "Share" feature (README, "Known leftovers").

Read more (official)

- Android Studio: https://developer.android.com/studio
- Run on a phone: https://developer.android.com/studio/run/device
- Run on the emulator: https://developer.android.com/studio/run/emulator
- Build from the command line: https://developer.android.com/build/building-cmdline
- App signing: https://developer.android.com/studio/publish/app-signing
- Gemini in Android Studio: https://developer.android.com/studio/gemini/overview
- Gemini data and privacy: https://developer.android.com/studio/gemini/data-and-privacy
- Use a local model: https://developer.android.com/studio/gemini/use-a-local-model
- Ollama documentation: https://docs.ollama.com
- Git documentation: https://git-scm.com/docs